

两个项目部署知识手册

(小白完全版 · 每条命令都有解释 · 每个操作都说理由)

项目一：铁路障碍物智能检测系统 —— 容器化部署与运维

项目二：服务器监控告警平台 —— Prometheus 全栈监控

环境：Rocky Linux 虚拟机 + Xshell 连接 + Windows 本机打包上传

技术栈：Docker · Docker Compose · Nginx · FastAPI · YOLOv8 · Vue3 · Prometheus · Grafana

目录

1. 基础概念：Docker 到底是什么（小白篇）
2. 项目一：整体架构与文件详解（每个文件是干嘛的）
3. 项目一：部署全流程，每条命令逐条讲解
4. 项目一：踩坑记录（6 个坑 + 排查思路 + 解决方案）
5. 项目二：监控基础概念（四大组件是什么）
6. 项目二：配置文件逐行详解
7. 项目二：部署、验证与告警测试
8. 项目二：踩坑记录（4 个坑 + 排查思路 + 解决方案）
9. 常用命令速查表
10. 面试问答速备

第 1 章 基础概念：Docker 到底是什么

1.1 为什么需要 Docker?

想象一个场景：你在自己的电脑上写好了一个网站，能跑起来。现在要把它放到公司的服务器上跑，结果报错——因为服务器上的 Python 版本不一样、缺一个库、系统不一样.....配置环境可能要折腾一整天。

Docker 解决的就是这个问题：它把「程序 + 它需要的所有环境（Python、依赖库、系统库）」一起打包成一个**镜像 (Image)**，这个镜像放到任何装了 Docker 的机器上都能原样跑起来。就像把整个「厨房连锅带碗」打包搬走，而不是只带菜谱。

为什么本项目要用 Docker? ① 环境一致：本地和服务器跑的是同一个打包环境；② 部署简单：一条 `docker compose up -d` 命令把前端+后端全启动；③ 好删好重来：容器删了就干净了，不污染系统。

1.2 几个核心词（必须搞懂）

名词	通俗解释	类比
镜像 (Image)	一个只读的「安装包」，包含了程序运行需要的一切	软件的 .exe 安装包
容器 (Container)	镜像跑起来之后的「实例」，互相隔离	双击安装包后正在运行的程序
Dockerfile	写「怎么打包」的说明书（一步步的指令）	安装包的制作配方
Docker Compose	一键编排多个容器的工具，用 yml 文件描述「要跑哪几个容器、怎么连」	一个总指挥：同时指挥前端、后端
数据卷 (Volume)	把容器里的文件夹「映射」到宿主机上，容器删了数据还在	把保险箱放容器外面，容器拆迁数据不丢
端口映射	把宿主机的端口转发到容器端口，如 "80:80"	前台电话 80 转接到内部 80 号分机

1.3 镜像和容器的关系（一句话记住）

镜像是模板，**容器**是运行实例。一个镜像可以同时跑 3 个容器，就像一个安装包可以装 3 台电脑。

1.4 多阶段构建是什么？为什么用？

前端项目（Vue3）需要 Node.js 来「编译打包」，但编译完之后，真正对外服务的只是几个静态文件（html/css/js），用 Nginx 托管就够了——**Node.js 编译完就没用了**。

多阶段构建就是在 Dockerfile 里写两个 FROM：第一个阶段用 Node 编译，第二个阶段只把编译产物拷贝进干净的 Nginx 镜像。这样最终镜像里没有 Node、没有源码，体积小、更安全。

为什么本项目后端用 CPU 版 PyTorch + slim 基础镜像？ 服务器没有独立显卡（无 NVIDIA GPU），装 GPU 版 PyTorch 会多拖约 2GB 的 CUDA 库，完全用不上；slim 是 Python 的精简版系统镜像（约 45MB），比完整版小得多。两者叠加让后端镜像从 4.16GB 降到约 2GB。

第 2 章 项目一：整体架构与文件详解

2.1 架构图（谁访问谁）



2.2 前端 Dockerfile 逐行解释

```
# ===== 第一阶段：用 Node 编译 =====
FROM node:20-alpine AS build # 基于 Node20 精简镜像(alpine=只有5MB的系统底座)，起名 build
WORKDIR /app # 容器内的工作目录设为 /app（后面的命令都在这执行）
COPY package*.json ./ # 只先拷贝依赖清单文件
RUN npm config set registry https://registry.npmmirror.com \
    && npm install # 用国内镜像源装依赖（不然从国外下载极慢/失败）
COPY . . # 再拷贝全部源码
RUN npm run build # 执行 vite build，产出 dist/ 静态文件

# ===== 第二阶段：用 Nginx 托管 =====
FROM docker.io/library/nginx:alpine # 干净的 Nginx 镜像（没有 Node）
COPY nginx.conf /etc/nginx/conf.d/default.conf # 用我们自己的 Nginx 配置
COPY --from=build /app/dist /usr/share/nginx/html # 只把编译产物拷进来
```

为什么先 COPY package.json 再 COPY 源码？ 利用 Docker 的**分层缓存**：只要 package.json 没变，npm install 这一层直接用缓存跳过（几秒），不用每次重装依赖。把变化少的放前面、变化多的放后面，是 Dockerfile 提速的核心技巧。

为什么用 npmmirror 源？ 默认的 npm 官方源在国外，国内服务器下载慢甚至超时。npmmirror（淘宝镜像）是国内同步源，几秒装完。

2.3 后端 Dockerfile 逐行解释

```
FROM python:3.10-slim                # Python3.10 精简系统底座

ENV PYTHONUNBUFFERED=1 \             # 日志实时输出（不缓冲），否则 docker logs 看不到实时日志
PYTHONDONTWRITEBYTECODE=1 \         # 不生成 .pyc 缓存文件，镜像更干净
PIP_NO_CACHE_DIR=1 \                # pip 不留下缓存，镜像更小
PIP_INDEX_URL=https://pypi.tuna.tsinghua.edu.cn/simple \      # 清华 PyPI 源（主源）
PIP_EXTRA_INDEX_URL=https://download.pytorch.org/whl/cpu \    # PyTorch 官方 CPU 源
                                (补充源)
PIP_TRUSTED_HOST=pypi.tuna.tsinghua.edu.cn                      # 信任该主机，避免 SSL 校验问题

RUN apt-get update && apt-get install -y --no-install-recommends \
    libgl1 \               # OpenCV 需要的图像库（没有它 import cv2 报错）
    libgl2.0-0 \          # OpenCV 需要的 C 库
    ffmpeg \              # 视频转码工具（修 H.264 播放问题的关键）
    && rm -rf /var/lib/apt/lists/* # 删掉安装包缓存列表，减小镜像体积

WORKDIR /app
COPY railway_obstacles_detection/runs/detect/train/weights/best.pt \
    /app/railway_obstacles_detection/.../best.pt # 先拷模型权重（几乎不变，吃缓存）

WORKDIR /app/railway-main
COPY backend/ ./backend/ # 再拷后端源码（经常改，放后面）
RUN pip install -r backend/requirements.txt # 装依赖（torch 走清华源+cpu补充源）

EXPOSE 8000 # 声明容器内监听 8000 端口
CMD ["uvicorn", "backend.app:app", "--host", "0.0.0.0", "--port", "8000"] # 启动命令
```

为什么 host 要写 0.0.0.0？ 如果写 127.0.0.1，只有容器自己能访问自己，外部（Nginx/用户）全都连不上。0.0.0.0 表示「监听所有网卡」，容器外的请求才进得来。

为什么用两个 pip 源？ 清华源是主源（日常包快），PyTorch 官方 CPU 源是补充源——清华源里也有 torch，但指定 extra-index 能确保拿到 +cpu 版本，避免误装巨大的 CUDA 版。

2.4 .dockerignore 是干嘛的

执行构建时，Docker 会把整个项目目录作为「构建上下文」发给 Docker 引擎。`.dockerignore` 列出「不要发」的内容：

```
node_modules/      # 前端依赖几百MB，容器里会重新 npm install，没必要传
dist/              # 历史编译产物，容器里会重新 build
railway_obstacles_datasets/  # 训练数据集几个GB，推理服务用不到
ultralitics/        # 训练框架源码副本，运行时用 pip 装的
outputs/            # 历史输出图片
```

理由： 构建上下文从几个 GB 降到几十 MB，构建从「传半天」变成秒级；同时防止数据集被打进镜像撑爆磁盘。

2.5 nginx.conf 关键配置逐行解释

```
server {
    listen 80;
    server_name _;

    # HTTP 强制跳转 HTTPS（用户输 http 也自动变 https）
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl;
    ssl_certificate      /etc/nginx/certs/nginx.crt;    # 自签证书（openssl 生成）
    ssl_certificate_key  /etc/nginx/certs/nginx.key;

    client_max_body_size 200m;    # 允许上传最大 200MB（默认才1MB，视频会传不上去）

    location / {                # 前端页面
        root   /usr/share/nginx/html;
        try_files $uri $uri/ /index.html;    # SPA路由回退：刷新页面不404
    }
    location /api/ {            # 接口请求 → 转发给后端容器
        proxy_pass http://backend:8000;    # backend 是 compose 里的服务名（容器名互联）
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
    location /api/ws/ {         # WebSocket → 也转发给后端
        proxy_pass http://backend:8000;
        proxy_http_version 1.1;    # WebSocket 必须 HTTP/1.1
        proxy_set_header Upgrade $http_upgrade;    # 升级协议头
        proxy_set_header Connection "upgrade";    # 保持长连接
        proxy_read_timeout 3600s;    # 1小时不断开（实时检测需要长连接）
    }
}
```

为什么摄像头必须 HTTPS? 浏览器安全规则：调用摄像头（getUserMedia）只允许在「安全上下文」——即 https:// 或 localhost。用 http://IP 访问会被浏览器直接拦截。配了自签证书的 HTTPS 后，点「高级→继续访问」即可正常调摄像头。

2.6 docker-compose.yml 关键配置逐行解释

```
services:
  backend:
    build: ./raliway-main          # 用这个目录里的 Dockerfile 构建镜像
    image: railway-yolo-backend:latest
    container_name: railway-backend
    restart: unless-stopped        # 容器崩了自动重启（除非人为 stop）
    expose: ["8000"]              # 只在 Docker 内网开放 8000，不映射到宿主机公网
    volumes:
      - ./data/outputs:/app/outputs # 数据卷：检测结果存宿主机，容器删了数据不丢
    healthcheck:                  # 健康检查：真正确认服务"就绪"而不只是"进程在"
      test: ["CMD", "curl", "-f", "http://localhost:8000/api/ping123"]
      interval: 15s               # 每15秒测一次
      timeout: 5s
      retries: 12                 # 最多容忍失败12次（模型预热约1分钟，要给足时间）
  frontend:
    build: ./frontend-main
    ports: ["80:80", "443:443"]   # 前端是唯一入口，映射到宿主机 80/443
    volumes:
      - ./certs/etc/nginx/certs:ro # 证书目录只读挂载
    depends_on:
      backend:
        condition: service_healthy # 关键：等 backend 真正 healthy 才启动前端
```

为什么后端 expose 而不 ports? 最小暴露原则：用户只需要访问前端(80/443)，API 由 Nginx 内网转发。后端不映射端口=公网扫不到它，攻击面小一条。

为什么需要 healthcheck? 没有 healthcheck 时，Docker 只知道「进程启动了」，不知道「服务真的能响应」。YOLO 模型加载要约1分钟，若前端立刻启动，用户第一时间访问会报错。healthcheck+depends_on 保证「后端真正就绪后前端才开门营业」。

第 3 章 项目一：部署全流程命令讲解

3.1 清理旧部署（每条命令为什么这么写）

```
docker compose down          # 优雅停止并删除本项目容器和网络。比直接删容器好：它会按 compose 记录清理干净
docker rm -f railway-backend # 强制删除指定容器（-f=先停再删）。兜底用，容器不存在也不报错(加了2>/dev/null)
docker rmi -f railway-backend:latest # 强制删除旧镜像，释放磁盘
docker images -qf dangling=true | xargs -r docker rmi # 删除"悬空镜像"(无标签的旧构建残留 <none>)
docker builder prune -f      # 清空构建缓存，旧缓存可能缓存了失败的层
rm -rf /root/实训            # 删除宿主主机上的旧项目目录
docker ps -a; docker images # 最终验证：确认没有残留
```

注意： 不要用 `docker system prune -a`！它会删掉所有没被容器使用的镜像——如果服务器上还有别的项目，会把别人的镜像也删了。

3.2 Windows 打包上传（本机 → 服务器）

```
# PowerShell 中执行
tar --exclude='*/node_modules' --exclude='*/dist' `
  --exclude='raliway-main/railway_obstacles_detection/ultralalytics' `
  --exclude='raliway-main/railway_obstacles_datasets' `
  --exclude='raliway-main/outputs' --exclude='*/.idea' `
  -czf railway-yolo.tar.gz docker-compose.yml frontend-main raliway-main
# tar = 打包命令；-c=创建包 -z=用gzip压缩 -f=指定文件名
# --exclude=排除大文件（数据集/依赖/历史输出），不然包会有几个GB

Get-Item railway-yolo.tar.gz | Select-Object @{N='SizeMB';E={[math]::Round($_.Length/1MB,1)}}
# 查看压缩包大小，确认排除生效（应约28MB）

scp railway-yolo.tar.gz root@192.168.12.128:/opt/
# scp = SSH 远程拷贝命令；格式：scp 本地文件 用户@IP:远程路径
# 注意：指定端口用 -P 22（大写P），端口不能写在路径里！
```

为什么要在服务器上校验压缩包？ scp 传大文件可能中途断开，得到半截包。解压会报 `gzip: stdin: unexpected end of file`。上传后先跑 `gzip -t 文件名`（t=test 完整性测试），提示完好再解压，能省掉大量排查时间。

3.3 服务器上解压与启动

```
mkdir -p /opt/railway-yolo/data/outputs # -p=连带父目录一起创建, 已存在不报错
tar -xzf /opt/railway-yolo.tar.gz -C /opt/railway-yolo # -x=解包 -z=gzip -C=指定目标目录

ls docker-compose.yml frontend-main raliway-main # 确认三个关键结构都在
ls -lh raliway-main/railway_obstacles_detection/runs/detect/train/weights/best.pt # 确认模型权重在

chcon -Rt svirt_sandbox_file_t /opt/railway-yolo/data/outputs
# Rocky默认开启SELinux(安全增强), 会给文件打"安全标签"。
# 容器默认只能读特定标签的目录, 不解绑会报 Permission denied。
# chcon -Rt = 递归修改目录标签为容器可访问类型

docker compose up -d --build
# up=创建并启动 -d=后台运行(detached) --build=先构建镜像再启动
# 首次约10-20分钟(下载基础镜像+装PyTorch); 之后有缓存1-2分钟
```

3.4 验证部署成功

```
docker compose ps # 看容器状态。backend 要显示 Up (healthy), frontend 显示 Up
docker compose logs --tail=50 backend # 看后端日志, 出现 "Application startup complete" 即成功
curl http://localhost/api/ping123 # 接口测试, 应返回 {"msg":"ok"}
curl -k https://localhost/ # -k=忽略自签证书校验, 测试 HTTPS 页面
```

3.5 生成自签 HTTPS 证书 (一行命令)

```
openssl req -x509 -nodes -days 3650 -newkey rsa:2048 \
    -keyout certs/nginx.key -out certs/nginx.crt -subj "/CN=192.168.12.128"
# req -x509 = 生成自签名X.509证书(不经过CA机构, 所以浏览器会提示"不安全")
# -nodes = 私钥不加密(加密码的话Nginx每次重启都要手动输密码)
# -days 3650 = 有效期10年
# -newkey rsa:2048 = 新建2048位RSA密钥对
# -subj "/CN=..." = 证书主体名, 填访问时用的IP或域名
```

为什么自签证书浏览器仍显示"不安全"? 因为证书不是权威 CA 机构签发的, 浏览器无法验证"你就是你"。但加密本身是有效的——这正是内网/测试环境用它解决摄像头权限的原因: 本质是让页面进入 HTTPS 安全上下文。

第 4 章 项目一：踩坑记录（6 个坑）

坑 1：解压报 `gzip: stdin: unexpected end of file`（压缩包损坏）

原因：scp 传输大文件中途断开，得到半截包；且 /opt 下残留了旧的坏包，误以为传成功了。

解决：上传后必做校验 `gzip -t 包名`；用 `ls -lh` 对比本地和服务器文件大小、看时间戳确认是新文件；坏了就删掉重传，或者只补传缺失的目录（如 backend）。

坑 2：scp 报 `realpath 22/opt/... No such file`（端口写法错误）

原因：把端口写进了路径：`scp -r xxx root@IP:22/opt/...`，scp 把 "22/opt/..." 当成了远程路径。

解决：scp 指定端口要用 `-P 22` 参数（大写 P）：`scp -P 22 -r 目录 root@IP:/opt/`

坑 3：构建失败 `pip subprocess-exited-with-error`（PyTorch 装不上）

原因：默认从国外 PyPI/官方源下载，服务器网络慢/不稳，超大包（torch 190MB）下载失败。

解决：Dockerfile 里设置环境变量换清华主源 + PyTorch CPU 官方源做补充源（见 2.3 节 ENV 部分），下载从"卡死"变"655KB/s 稳定拉完"。

坑 4：构建报 `Could not open requirements.txt`（backend 目录是空的）

原因：压缩包损坏解压中断，backend 目录只建了空壳没解出文件。当时只检查了目录存在、没检查文件数量。

解决：`ls -la backend/` 确认 5 个源码文件齐全；缺失就用 scp -r 只补传 backend 小目录（比整个包重传快得多）。

经验：验证解压结果不要只看"目录在不在"，要数"文件对不对"。

坑 5：视频检测后左侧视频黑屏放不出来（功能性 Bug）

原因：后端 OpenCV 用 `avc1`（H.264）编码写视频，但 Linux 下 OpenCV 自带的 ffmpeg 不含 **x264 编码器**，VideoWriter 静默失败产出空文件。Windows 下有系统编码器所以本地好好的。

解决：先以 `mp4v`（OpenCV 自带）写中间文件，处理完调用**系统 ffmpeg** 转码成 H.264 + yuv420p + faststart（浏览器通用、支持边下边播）；Dockerfile 加装 ffmpeg；代码里增加 `out.isOpened()` 检查，失败直接报 500 而不是返回空文件。

坑 6：摄像头被浏览器拦截（getUserMedia 报错）

原因：浏览器只允许 HTTPS 或 localhost 调用摄像头，http://IP 是不安全上下文，直接拦截。

解决：给 Nginx 配自签证书启用 HTTPS（见 3.5），HTTP 301 跳转 HTTPS，摄像头直接可用，还顺带加密了传输。

第 5 章 项目二：监控基础概念

5.1 为什么要监控？

服务器半夜 CPU 爆了、磁盘满了、服务挂了——没人盯着就不知道，等用户报障就晚了。监控系统的作用：**持续采集服务器指标** → **可视化展示** → **超过阈值自动发通知**。这是运维岗位最核心的日常工作之一。

5.2 四个组件是什么（类比快递系统）

组件	干什么	类比
Node Exporter	安装在服务器上的"传感器"，把 CPU/内存/磁盘/网络数据暴露在 /metrics 接口上	快递柜（存放原始数据）
Prometheus	"拉取者"：每 15 秒主动去各 Exporter 抓数据存进自己的时序数据库，并按规则判断是否告警	巡查员（定时抄表+判断异常）
AlertManager	接收 Prometheus 发来的告警，去重、分组、延迟合并后发邮件	报警前台（汇总后统一通知）
Grafana	可视化面板，把 Prometheus 里的数据画成图表大屏	监控电视墙（好看直观）

为什么 Prometheus 是"拉"模型（主动抓）而不是"推"？ 拉模型让 Prometheus 是唯一调度方：抓不到某个目标就知道它挂了（天然健康检查）；新增采集目标只需在配置里加一行，架构简单可控。

5.3 数据流转一句话

Node Exporter 暴露指标 → Prometheus 每 15s 抓取并存储 → 每15s评估告警规则(表达式超阈值持续N分钟就触发) → 告警推给 AlertManager → 分组/等10s合并 → 发邮件 → Grafana 随时查库画图。

第 6 章 项目二：配置文件逐行详解

6.1 prometheus.yml (主配置)

```
global:
  scrape_interval: 15s      # 每15秒抓一次指标（太频繁伤性能，太稀疏丢细节）
  evaluation_interval: 15s  # 每15秒评估一遍告警规则

rule_files:
  - "rules/*.yaml"         # 加载告警规则文件

alerting:
  alertmanagers:
    - static_configs:
        - targets: ['alertmanager:9093']  # 告警发给谁（容器名=主机名，Docker内网自动DNS）

scrape_configs:
  - job_name: 'prometheus'    # 监控 Prometheus 自己（自监控）
    static_configs: [{ targets: ['localhost:9090'] }]
  - job_name: 'node-exporter' # 监控宿主机
    static_configs:
      - targets: ['node-exporter:9100']  # 服务名:端口，compose网络内自动解析
        labels: { instance: 'rocky-linux-server' } # 给这台机器起个可读名字
```

6.2 alerts.yml (告警规则, 每条 PromQL 都解释)

```
- alert: HighCPUUsage          # 规则名: CPU过高
  expr: 100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
  # node_cpu_seconds_total: CPU每种状态累计运行秒数(计数器只增不减)
  # rate(...[5m]): 5分钟窗口内每秒增量 = CPU每秒使用了多少秒
  # {mode="idle"}: 只看"空闲"状态 → 空闲的减少量 = 被使用的量
  # 100 - 空闲% = 使用率%。avg by(instance)按机器求平均(多核)
  for: 2m                      # 持续超过2分钟才算告警(防止瞬时抖动误报)
  labels: { severity: warning }
  annotations:                 # 告警内容模板, $value是当前值
    summary: "服务器 {{ $labels.instance }} CPU 使用率过高"

- alert: HighMemoryUsage
  expr: (1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100 > 85
  # 可用内存/总内存=可用比例; 1-可用比例=使用比例

- alert: LowDiskSpace
  expr: (node_filesystem_avail_bytes{fstype!~"tmpfs|overlay"} /
    node_filesystem_size_bytes{fstype!~"tmpfs|overlay"}) * 100 < 20
  # fstype!~"tmpfs|overlay": 排除内存盘和容器 overlay 文件系统, 只看真实磁盘

- alert: HostDown
  expr: up{job="node-exporter"} == 0
  # up 是 Prometheus 内置指标: 抓取成功=1, 失败=0。抓不到=宕机
```

6.3 alertmanager.yml (告警分发配置)

```
global:
  smtp_smarthost: 'smtp.qq.com:465'      # QQ邮箱的SMTP服务器:端口(465=SSL加密)
  smtp_from: 'xxx@qq.com'                # 发件人
  smtp_auth_username: 'xxx@qq.com'        # 认证账号
  smtp_auth_password: '16位授权码'       # 注意: 不是QQ密码! 是邮箱设置里生成的SMTP授权码
  smtp_require_tls: false                 # 465端口本身是SSL, 无需再STARTTLS

route:
  # 路由树: 什么告警发给谁
  group_by: ['alertname']                 # 同名告警分到一组
  group_wait: 10s                          # 第一条告警先等10秒, 把同组的一起发(防轰炸)
  group_interval: 1m                       # 同组新告警最短间隔1分钟
  repeat_interval: 1h                     # 同一告警未恢复, 每1小时重发提醒
  receiver: 'email-notify'                # 默认接收器

receivers:
  - name: 'email-notify'
    email_configs:
      - to: 'xxx@qq.com'
        send_resolved: true               # 恢复后也发[RESOLVED]邮件(知道问题解决了)

inhibit_rules:
  # 抑制规则
  - source_match: { severity: 'critical' }
    target_match: { severity: 'warning' }
    equal: ['instance']                   # 同一台机器上, critical触发时屏蔽warning(如宕机时不必再看磁盘告警)
```

6.4 Grafana 数据源自动配置 (provisioning)

```
# grafana/provisioning/datasources/datasource.yml
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    access: proxy                          # 由Grafana后端代为请求(浏览器不直连Prometheus)
    url: http://prometheus:9090           # 容器名互联
    isDefault: true                       # 设为默认数据源
```

为什么用 provisioning 文件而不是网页里手动添加? 配置即代码: 容器重建后数据源自动恢复, 不用人工重新点; 这也是“基础设施即代码(IaC)”思想的入门实践。

第 7 章 项目二：部署、验证与告警测试

7.1 上传与启动

```
# Windows PowerShell: 整个目录传上去
scp -P 22 -r monitoring root@192.168.12.128:/opt/

# 服务器：修改邮箱配置后启动
vi /opt/monitoring/alertmanager/alertmanager.yml # 改SMTP为自己的邮箱+授权码
cd /opt/monitoring
docker compose up -d          # 拉取4个镜像并启动，约5-10分钟
docker compose ps             # 4个容器都应为 Up
```

7.2 验证采集是否正常（注意端口没暴露到宿主机）

```
curl http://localhost:9090/... # X 报错！因为compose里Prometheus用的是expose(仅内网)
docker exec monitor-grafana curl -s http://prometheus:9090/api/v1/targets | head -c 800
# ✓ 正确姿势：借用grafana容器在Docker内网里查。看到 "health":"up" 即采集正常
```

为什么故意不把 9090/9100/9093 映射到宿主机？ 最小暴露原则：只有 Grafana(3000) 需要给人看，其余组件都留在内网，外部攻击面最小。运维常用「借另一个容器的网络」来调试不暴露端口的服务。

7.3 Grafana 导入主机大屏

```
浏览器打开 http://192.168.12.128:3000 # admin/admin123 登录
Dashboards → New → Import → 输入ID 1860 → Load → 数据源选Prometheus → Import
# 1860 = 社区最著名的 Node Exporter Full 面板，90+个主机指标图表
```

7.4 告警实测（压测 → 触发 → 邮件 → 恢复）

```
# 1) 起4个死循环吃满CPU（机器2核，4个循环必超80%阈值）
for i in 1 2 3 4; do while :; do :; done & done

# 2) 等3-4分钟（规则for:2m + 评估周期），查告警状态
docker exec monitor-grafana curl -s http://prometheus:9090/api/v1/alerts | grep -o '"state":'
[a-z]*"

#   pending=已超阈值未满足持续时间；firing=已触发

# 3) 收到[FIRING]邮件后，杀掉压测进程
ps aux --sort=-%cpu | head -8    # 按CPU排序找进程，死循环的cmdline是bash
kill -9 <PID1> <PID2> <PID3> <PID4>

# 4) 几分钟后收到[RESOLVED]恢复邮件 → 告警生命周期闭环
```

为什么 pkill -f "while :" 杀不掉？ pkill -f 只匹配进程命令行(cmdline)，而这几个死循环是bash后台子进程，cmdline只显示"bash"——"while"只是bash内部语法不在命令行里。正确做法：ps 找PID 后 kill，或回到启动它的窗口 kill %1 %2 %3 %4。

第 8 章 项目二：踩坑记录（4 个坑）

坑 1：Grafana 所有下拉框没值，图表全 No data

排查：① `docker exec monitor-grafana curl -s -u admin:admin123`

`http://localhost:3000/api/datasources` 返回 `[]` → Grafana 里根本没有数据源；② `ls /etc/grafana/provisioning/datasources/` 报 `Permission denied`。

原因：scp 从 Windows 传目录权限过严，Grafana 进程(uid 472)读不了挂载的 provisioning 目录，`datasource.yml` 没被加载。

解决：宿主机 `chmod -R 755 /opt/monitoring/grafana` + `docker compose restart grafana`。兜底方案：直接调 Grafana API POST 创建数据源。

经验：容器读不到宿主机挂载文件，先查「属主权限」再查「SELinux 标签」。

坑 2：告警触发但邮件收不到——lookup smtp.qq.com: no such host

排查：`docker logs monitor-alertmanager | grep -i error` 看到 DNS 解析的主机名是 `smtp.qq.com`（多打了个@）。

解决：grep 检查配置文件，把 `smtp_smarthost` 改回 `'smtp.qq.com:465'`，restart alertmanager，重新压测后邮件到达。

经验："notify retry canceled"后面跟着的 err 才是真正原因；配置类问题先 grep 原文，别靠记忆。

坑 3：杀掉压测进程后 CPU 回落但 Load 还显示很高

原因：Load average 是 1/5/15 分钟滑动平均，天生滞后，进程死亡后也要几分钟缓慢衰减，属正常现象。

判断：看实时 CPU（已回落 4%）而不是 Load；Load 过几分钟自动归位。Grafana 图表上的压测尖峰会保留到数据过期（15天），这也是事故回溯的依据。

坑 4：宿主机 curl localhost:9090 无任何输出

原因：compose 中 Prometheus 用的是 `expose`（仅容器网络内开放），没有映射宿主机端口；
`curl -s` 把"连接拒绝"静默了。

解决：进容器网络里测：`docker exec monitor-grafana curl http://prometheus:9090/...`。这是"安全配置导致调试路径变化"的典型案例。

第 9 章 常用命令速查表

9.1 Docker / Compose 日常

命令	作用
<code>docker compose up -d</code>	后台启动所有服务
<code>docker compose up -d --build</code>	重新构建镜像并启动（改了代码后用）
<code>docker compose down</code>	停止并删除容器/网络（数据卷保留）
<code>docker compose ps</code>	查看服务状态（healthy/Exited）
<code>docker compose logs -f --tail=100 服务名</code>	实时追踪某服务日志
<code>docker compose restart 服务名</code>	重启某服务
<code>docker ps -a</code>	所有容器（含已停止）
<code>docker images</code>	本地镜像列表
<code>docker rmi -f 镜像名</code>	删除镜像
<code>docker exec -it 容器名 sh</code>	进入容器内部排查
<code>docker system df</code>	查看 Docker 磁盘占用
<code>systemctl enable --now docker</code>	Docker 开机自启并立即启动

9.2 Linux 排障

命令	作用
<code>ps aux --sort=-%cpu head</code>	按CPU排序找高占用进程
<code>top -b -n1 head -15</code>	一次性输出进程快照
<code>kill -9 PID</code>	强制杀进程（先确认进程身份）
<code>curl -I URL</code>	只看HTTP响应头
<code>curl -k https://IP</code>	忽略自签证书测试
<code>ss -tlnp grep :80</code>	查看端口被谁监听
<code>firewall-cmd --add-port=80/tcp --permanent && firewall-cmd --reload</code>	Rocky 放行端口
<code>getenforce / setenforce 0</code>	查SELinux状态 / 临时关闭
<code>chcon -Rt svirt_sandbox_file_t 目录</code>	给目录打容器可访问的SELinux标签
<code>chmod -R 755 目录</code>	放开目录读权限
<code>gzip -t 文件</code>	校验gzip压缩包完整性
<code>tar -czf / -xzf</code>	打包压缩 / 解包
<code>scp -P 22 -r 本地 用户@IP:路径</code>	远程拷贝（大写P指定端口）
<code>docker logs 容器 2>&1 grep -i error</code>	过滤容器错误日志

第 10 章 面试问答速备

10.1 项目一相关

问题	回答要点
为什么用 Docker?	环境一致性（一次构建到处运行）、隔离性、部署可复现、秒级回滚、资源占用比虚拟机小
多阶段构建解决什么?	编译工具(Node)不进生产镜像 → 镜像小、攻击面小、拉取快。前端从 Node+源码 缩到纯 Nginx+静态文件
healthcheck 的意义?	区分"进程启动"和"服务就绪"。YOLO模型预热1分钟，靠 healthcheck+depends_on 保证前端在后端真正就绪后才启动
为什么后端不映射公网端口?	最小暴露原则，外部只能经 Nginx 反代访问，后端漏洞不直接暴露
Nginx 反代 WebSocket 和普通 HTTP 区别?	需 HTTP/1.1 + Upgrade/Connection 头完成协议升级 + 加大 read_timeout 防长连接被掐
数据卷解决什么?	容器是"用完即扔"的，检测结果挂载到宿主机后容器重建数据不丢
镜像怎么瘦身的?	slim底座 + CPU版torch(省约2GB) + .dockerignore缩小上下文 + 合并RUN层删缓存 + 多阶段构建
视频黑屏怎么排查的?	现象→查返回文件大小为0→定位 VideoWriter 静默失败→根因是OpenCV内置 ffmpeg无x264→系统ffmpeg转码H.264→加 isOpened() 防御

10.2 项目二相关

问题	回答要点
Prometheus 工作原理?	拉模型: 定时抓取各目标/metrics → 存TSDB → 定期评估告警规则 → 触发推给AlertManager分发
PromQL 写个CPU使用率?	$100 - \text{avg by(instance)}(\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{"idle"}\}[5\text{m}])) * 100;$ rate算计数器每秒增量, idle减少量即使用量
告警怎么防风暴/误报?	for持续时间防抖、group_by+group_wait合并、repeat_interval限重发、critical抑制同实例warning、路由分级
up 指标是什么?	Prometheus内置: 抓取成功1/失败0, 用 up==0 做存活告警
Grafana 图表 No data 怎么排查?	①查数据源(/api/datasources) ②查Prometheus targets健康度 ③查仪表盘变量(datasource/job/instance)是否选中 ④查采集器本身存活
邮件告警收不到怎么排查?	看AlertManager日志的err字段 (本次是SMTP主机名打错DNS解析失败) → 修正配置 → 重启 → 重放告警验证闭环
线上CPU飙高怎么排查?	top/ps --sort=-%cpu 定位PID → 确认进程身份 → kill或分析 → 监控确认回落; 注意load average是滑动平均有滞后
容器读不到挂载目录?	先查属主权限(ls -la, chmod 755), 再查SELinux标签(chcon -Rt svirt_sandbox_file_t), 最后才考虑关SELinux

10.3 简历项目描述（浓缩版）

项目一：铁路障碍物智能检测系统 —— 容器化部署与运维

(Docker/Compose/Nginx/FastAPI/YOLOv8/Vue3/Rocky Linux)

- 前后端分离项目容器化改造，compose 一键部署；多阶段构建+slim+CPU版PyTorch，镜像 4.16GB→约2GB
- Nginx 反代：API转发、WebSocket透传、200M上传、SPA回退、HTTP跳HTTPS(自签证书)；后端端口不对外暴露
- healthcheck 编排启动顺序、数据卷持久化、分层缓存二次构建<1分钟；解决 OpenCV 无 H.264 编码器、scp 传输损坏、SELinux 等问题

项目二：服务器监控告警平台 —— Prometheus 全栈监控

(Prometheus/Grafana/AlertManager/Node Exporter/Docker Compose)

- 一键部署四组件监控体系，CPU/内存/磁盘/网络 7×24 采集与可视化大屏 (Node Exporter Full)
- 6 类分级告警规则 + critical 抑制 warning 防告警风暴；邮件通知 FIRING→RESOLVED 全生命周期实测闭环
- 排障：provisioning 目录权限、SMTP 配置错误 DNS 失败、pkill 匹配原理、Load 均值滞后等